

PRACTICAL NO. 2

NAME : BHOSALE SHREYAS UDDHAV

PRN : 202501070125

BATCH : E21

2.1.1. LIST OPERATIONS

Problem Statement:

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- Add: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- Remove: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- Display: Displays the current list of integers. If the list is empty, display "List is empty".
- Quit: Exits the program.

The program should handle invalid choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code:

```
# write the code...
l1=[]
while True:
    print('1. Add')
    print('2. Remove')
    print('3. Display')
    print('4. Quit')
    n=input('Enter choice: ')
    if n==1:
        try:
            l1.append(int(input('Integer: ')))
            print('List after adding:', l1)
        except:
            pass
    elif n==2:
        while len(l1)>0:
            if n==2:
                remove=int(input('Integer: '))
                if remove not in l1:
                    print('Element not found')
                else:
                    l1.remove(remove)
                    print('List after removing:', l1)
            else:
                break
    elif n==3:
        print(l1)
    elif n==4:
        break
    else:
        print('Invalid choice')
```

```

        if len(l1)==0:
            print('List is empty')
        else:
            print(l1)
    elif n==4:
        break
    else:
        print('Invalid choice')

```

Output:

Average time
0.100 s
100.33 ms

Maximum time
0.143 s
143.00 ms

2 out of 2 shown test case(s) passed
 1 out of 1 hidden test case(s) passed

Test case 1

100 ms

Debug

Expected output

1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 1
 Integer: 11
 List after adding: [11]
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 1
 Integer: 25
 List after adding: [11, 25]
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 2
 Integer: 25
 Element not found
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 2
 Integer: 11
 List after removing: [25]
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 2
 Integer: 25
 List after removing: []
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 3
 List is empty
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 2
 List is empty

Actual output

1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 1
 Integer: 11
 List after adding: [11]
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 1
 Integer: 25
 List after adding: [11, 25]
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 2
 Integer: 25
 Element not found
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 2
 Integer: 11
 List after removing: [25]
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 2
 Integer: 25
 List after removing: []
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 3
 List is empty
 1. Add
 2. Remove
 3. Display
 4. Quit
 Enter choice: 2
 List is empty

< Prev

Reset

Submit

Next >

Average time

0.100 s

100.33 ms

Maximum time

0.143 s

143.00 ms

2 out of 2 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1

Test case 2

Debug

Expected output	Actual output
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 1	Integer: 1
List after adding: [1]	List after adding: [1]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 2	Integer: 2
List after adding: [1, 2]	List after adding: [1, 2]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 3	Integer: 3
List after adding: [1, 2, 3]	List after adding: [1, 2, 3]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 4	Integer: 4
List after adding: [1, 2, 3, 4]	List after adding: [1, 2, 3, 4]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 2	Integer: 2
List after removing: [1, 3, 4]	List after removing: [1, 3, 4]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 6	Integer: 6
Element not found	Element not found
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit

< Prev

Reset

Submit

Next >

2.1.2. Dictionary Operations

Problem Statement:

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion - Insert a new key-value pair into the dictionary using user input.
2. Update - Update the value of an existing key by using user input.
3. Deletion - Delete a specified key from the dictionary using user input.
4. Traverse - Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert the new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform updates only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/output.

Code:

```
# Initial dictionary with 10 predefined records
student = {
    1: 'Amit',
    2: 'Riya',
    3: 'Kiran',
    4: 'Neha',
    5: 'Arjun',
    6: 'Pooja',
    7: 'Rahul',
    8: 'Sneha',
    9: 'Vikram',
    10: 'Anjali'
}
print('Original Dictionary:', student)
key=int(input())
value=input()
student.update({key: value})
print('After Insertion:', student)
key=int(input())
value=input()
if key in student.keys():
    student.update({key: value})
print('After Update:', student)
key=int(input())
if key in student.keys():
    student.pop(key)
print('After Deletion:', student)
print('Traversing Dictionary:')
for key, value in student.items():
```

```
print(key, ': ', value)
```

Output:

Average time
0.029 s
29.25 ms

Maximum time
0.034 s
34.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 **Pass** **100%** **100%**

Expected output

Original Dictionary: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali')
11
Suresh
After Insertion: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh')
3
Karthik
After Update: (1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh')
5
After Deletion: (1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh')
Traversing Dictionary:
1: Amit
2: Riya
3: Karthik
4: Neha
6: Pooja
7: Rahul
8: Sneha
9: Vikram
10: Anjali
11: Suresh

Actual output

Original Dictionary: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali')
11
Suresh
After Insertion: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh')
3
Karthik
After Update: (1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh')
5
After Deletion: (1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh')
Traversing Dictionary:
1: Amit
2: Riya
3: Karthik
4: Neha
6: Pooja
7: Rahul
8: Sneha
9: Vikram
10: Anjali
11: Suresh

Test case 2 **Pass** **100%** **100%**

Expected output

Original Dictionary: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali')
12
Meera
After Insertion: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera')
6
Divya
After Update: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera')
1
After Deletion: (2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera')
Traversing Dictionary:
2: Riya
3: Kiran
4: Neha
5: Arjun
6: Divya
7: Rahul
8: Sneha
9: Vikram
10: Anjali
12: Meera

Actual output

Original Dictionary: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali')
12
Meera
After Insertion: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera')
6
Divya
After Update: (1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera')
1
After Deletion: (2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera')
Traversing Dictionary:
2: Riya
3: Kiran
4: Neha
5: Arjun
6: Divya
7: Rahul
8: Sneha
9: Vikram
10: Anjali
12: Meera

< Prev

Reset

Submit

Next >

2.2.1. Linear search Technique

Problem Statement:

Write a program to check whether the given element is present or not in the array of elements, using linear search.

Input Format:

- The first line of input contains the array of integers which are separated by space.
- The last line of input contains the key element to be searched.

Output Format:

- If the element is found, print the index. If the element is found multiple times, print the starting index.
- If the element is not found, print 'Not Found'.

Sample Test Case:

Input: 1 2 4 3 5 0

3

Output: 2

Code:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i]==key:
            return i
    return -1

arr=list(map(int,input().split()))
key=int(input())

result=linear_search(arr, key)
if result != -1:
    print(result)
else:
    print('Not Found')
```

Output:

CTP1709...

Submit

Average time
0.018 s
17.75 ms

Maximum time
0.026 s
26.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1
Expected output
1 2 3 4 3 5 6
3
2

Actual output
1 2 3 4 3 5 6
3
2

Test case 2
Expected output
1 2 3 4 5 6 7 8 9 19
28
Not found

Actual output
1 2 3 4 5 6 7 8 9 19
28
Not found

2.2.2. Captain of the Team

Problem Statement:

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

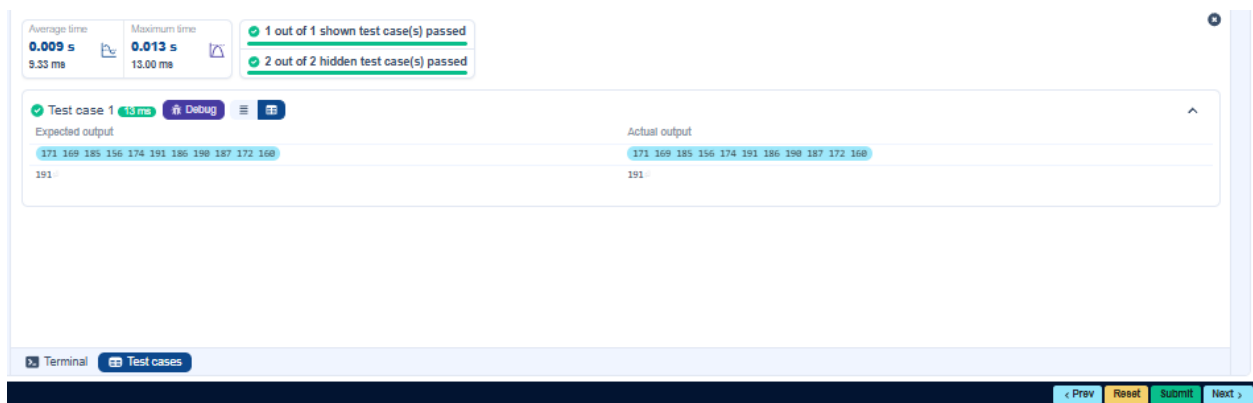
Output Format:

The output should be the height (in centimeters) of the tallest player.

Code:

```
heights=list(map(int,input().split()))
tallest_player=max(heights)
print(tallest_player)
```

Output:



The screenshot displays a code execution environment with the following details:

- Performance Metrics:**
 - Average time: 0.009 s (9.33 ms)
 - Maximum time: 0.013 s (13.00 ms)
- Test Results:**
 - 1 out of 1 shown test case(s) passed
 - 2 out of 2 hidden test case(s) passed
- Test Case 1 Details:**
 - Expected output: 171 169 185 156 174 191 186 190 187 172 160
 - Actual output: 171 169 185 156 174 191 186 190 187 172 160
 - 191
- Interface Elements:**
 - Buttons: < Prev, Reset, Submit, Next >
 - Terminal and Test cases tabs are visible at the bottom.